52. Linux Interview and Oncall Drills

52. Linux Interview and Oncall Drills

Phase: Phase 10 — Observability and troubleshooting
Estimated time: 2 days
Slug: `52-interview-drills`

Learning objectives

- Explain debugging aloud with structure
- Solve timed scenarios
- Write clear handoff notes
- Prioritize user impact

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Interviews and real incidents test the same thing: can you stay structured when the situation is vague and urgent? Someone says “the site is slow” and looks at you. Weak responders jump to a favorite theory, change three things, and cannot say which one mattered. Strong responders spend the first ten minutes on a repeatable routine: **define the scope, gather evidence**, and only then pick a hypothesis. The routine is the skill, and you can practice it.

The first ten minutes are: define the problem and its scope, gather evidence, communicate. One compact way to remember it is define-scope-gather-evidence. Fixing comes after you know what is broken and for whom.

Core concepts

Step	Question	Example
Define	what exactly is failing? what is expected?	“login returns 500 since 14:05”
Scope	who and what is affected?	one region, all users, one host
Evidence	what do logs, metrics, and state say?	error rate, load, disk, DNS
Hypothesize	what single cause fits all evidence?	full disk on db host
Change	smallest reversible action	clear logs, restart one unit
Verify	did the symptom end, for everyone?	health check, user report
Communicate	status, owner, next update time	message every 15 minutes

```
alert/report
    |
    v
DEFINE (symptom, start time, expected)
    |
    v
SCOPE (who? where? since when? what changed?)
    |
    v
EVIDENCE (uptime, df, free, ss, journalctl, metrics)
    |
    v
hypothesis -> smallest fix -> VERIFY -> write down -> prevent
      ^                          |
      +---- not fixed? back to evidence, do not guess
```

Worked examples

Example 1: “The site is slow.” First ten minutes at the shell.

```
uptime                      # load vs CPU count
nproc
free -h                     # memory pressure, swap use
df -h; df -i                # space and inodes
vmstat 1 5                  # run queue, swap, io wait
ss -s                       # socket totals
sudo journalctl -p err -n 30 --no-pager
```

Say aloud what each result rules in or out.

Example 2: “Disk full at 2am.”

```
df -h
sudo du -xh / --max-depth=2 2>/dev/null | sort -h | tail
sudo lsof +L1 | head        # deleted files still held open
```

Deleted-but-open files explain “df says full, du says not”.

Example 3: “TLS expired.”

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/
null | openssl x509 -noout -enddate
```

Example 4: “SSH refused suddenly.”

```
nc -vz host 22
sudo systemctl status ssh
sudo ss -tlnp | grep :22
sudo nft list ruleset | grep -n 22
```

Example 5: a handoff note template.

```
Status: mitigated at 02:40, monitoring
Impact: checkout errors, ~12 min, 8 percent of requests
Evidence: db disk 100 percent, error "no space left"
Action: rotated logs, freed 6 GB, restarted writer
Next: root cause for log growth, owner: ana, due Friday
```

Mini-lab

1. Pick one scenario from the examples. Set a 10-minute timer.
2. Write the problem statement in one sentence before touching a command.
3. List three scope questions you would ask a stakeholder.
4. Run only read-only commands and paste the output into notes.
5. State one hypothesis and the single command that would disprove it.
6. Practice a status update: what we know, what we are doing, next update time. Say it out loud in under 30 seconds.
7. Swap with a friend and break each other’s lab VM with a hidden fault.

Decision guide / failure modes

Situation	Move
No idea where to start	define scope, run the baseline commands
Two plausible causes	pick the check that separates them cheaply
Pressure to restart	capture evidence first, then restart
Fix applied	verify from the user’s point of view
Incident over	note timeline, cause, prevention

Failure modes: exotic theories before disk, DNS, and memory; silence while working; changing multiple things at once; declaring victory with no verification; and no notes, so

the next shift repeats your work. Score yourself on structure, evidence, communication, and verification.

Common mistakes

- Jumping to exotic theories first
- No communication during incident
- Fixing without verifying

Exercises

Work in `exercises/52-interview-drills/`. Read that folder's `README.md` fully before coding.

```
cd exercises/52-interview-drills
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `incident_first_10m`. It prints a single hyphenated token that summarizes, in three ordered verbs or nouns, what to do in the first ten minutes of an incident before proposing a fix. Use lowercase and hyphens only, one line, no spaces. After it passes, run a timed drill from the lesson: choose a scenario, write the one-sentence problem, list scope questions, and run only read-only commands. Save the transcript and a handoff note in the format shown in Example 5. Rate yourself from 1 to 5 on structure, evidence, communication, and verification.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/52-interview-drills/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. First 3 checks for 'slow'?
2. Why verify after fix?
3. Handoff note contents?

Further reading

- https://sre.google/sre-book/managing-incidents/